TESSAN

RECRUTEMENT

TEST TECHNIQUE

Stage Développeur Fullstack / Mobile

Une mini salle d'attente de téléconsultation : une petite API et une app mobile React Native qui affiche la position du patient dans la file, mise à jour automatiquement.

3 h
de travail maximum

72 h
pour rendre le projet après réception

15 min
de débrief pendant la visio technique

01 Contexte

Tessan développe des solutions de téléconsultation installées en pharmacie, et une app mobile patient qui accompagne la santé au quotidien. Le stage porte sur cette app, développée en React Native / TypeScript, et sur son lien avec les API de la plateforme.

Ce test reproduit en petit une situation réelle du poste : une app mobile qui consomme une API, avec des états réseau à gérer proprement. **Aucun piège.** Ce qui compte : un code qui marche, un code lisible, et la capacité à l'expliquer.

02 Le sujet : la salle d'attente de téléconsultation

Avant une téléconsultation, le patient attend qu'un médecin se libère. L'objectif : construire une mini salle d'attente, avec une petite API et un écran mobile qui affiche la position dans la file, mise à jour automatiquement.

Parcours attendu

1 Choisir un motif de consultation dans une liste.

2 Rejoindre la file d'attente.

3 Suivre sa position et le temps d'attente estimé, rafraîchis automatiquement.

4 Voir clairement à l'écran quand c'est son tour.

5 Quitter la file à tout moment, avec une confirmation.

Vue d'ensemble



> Toutes les données sont **fictives** et gardées en mémoire : pas de base de données, pas de compte utilisateur, aucune information personnelle.

03 Partie 1 : l'API

Une API REST minimale, données en mémoire. Stack libre : Node.js / TypeScript (Express, Fastify, NestJS) ou .NET. Le .NET est un plus, sans être exigé.

Méthode	Route	Rôle
GET	`/reasons`	Liste des motifs de consultation (exemples : fièvre, renouvellement d'ordonnance, douleur, autre)
POST	`/tickets`	Rejoindre la file avec un motif, renvoie le ticket créé
GET	`/tickets/:id`	État du ticket : statut, position, attente estimée
DELETE	`/tickets/:id`	Quitter la file

Règles métier

- Trois statuts : `waiting` en attente, `called` c'est son tour, `cancelled` a quitté la file.
- Au démarrage, la file contient déjà 3 patients fictifs, pour qu'un nouveau ticket ne soit pas appelé tout de suite.
- La file avance toute seule : toutes les 20 secondes, le premier ticket en attente passe en `called` et les suivants avancent d'une place.
- L'attente estimée est calculée par l'API à partir de la position et d'une durée moyenne par consultation, paramétrable (5 minutes par défaut).
- Erreurs explicites : **400** si le motif est absent ou inconnu, **404** si le ticket n'existe pas, **409** si l'on quitte un ticket déjà appelé ou annulé.

Exemple de réponse de `GET /tickets/:id`

```
{
  "id": "a1b2c3",
  "reason": "fievre",
  "status": "waiting",
  "position": 3,
  "estimatedWaitMinutes": 15
}
```

04 Partie 2 : l'app mobile

React Native avec TypeScript, Expo recommandé pour aller vite. Sans aucune expérience React Native, une app React web est acceptée : le préciser dans le README.

ÉCRAN 1

Motif

- Liste des motifs chargée depuis l'API.
- Sélection d'un motif.
- Bouton « Rejoindre la file ».

ÉCRAN 2

Salle d'attente

- Position et attente estimée, rafraîchies toutes les 10 à 15 secondes.
- Un message clair quand le statut passe à `called`.
- Bouton « Quitter la file » avec confirmation.

Ce qui pèse le plus dans l'évaluation

États de chargement
Premier chargement visible, rafraîchissements discrets.

Erreurs
API coupée ou lente : un message compréhensible et un bouton « Réessayer », sans planter.

Rafraîchissement maîtrisé
Il s'arrête quand l'écran est quitté ou quand le ticket est appelé ou annulé.

Types TypeScript
Pour les données échangées avec l'API.

Au moins 2 tests automatisés
Côté app ou côté API, sur une logique réelle (pas un test qui vérifie que 1 + 1 = 2).

05 Bonus facultatifs

> À faire seulement si le socle est solide. **Un bonus ne compense pas un socle fragile.**

- Reprise après fermeture de l'app : le ticket en cours est retrouvé (stockage local).
- Notification locale quand c'est son tour.
- Accessibilité : libellés pour lecteur d'écran, tailles de texte dynamiques.
- Temps réel sans polling (WebSocket ou Server-Sent Events).
- Une CI simple qui lance les tests (GitHub Actions).
- Un Dockerfile pour l'API.

06 Règles du jeu

3 heures maximum
Un socle solide avec un README honnête sur ce qui manque vaut mieux qu'un rendu complet mais fragile.

IA autorisée
Copilot, ChatGPT, Claude ou autre, à deux conditions : indiquer dans le README les outils utilisés et pour quoi faire, et pouvoir expliquer chaque ligne du rendu pendant le débrief.

Données fictives uniquement
Aucun nom réel, aucune information de santé réelle, aucun secret dans le dépôt.

Librairies libres
Une ligne dans le README pour justifier celles qui structurent le projet.

Pas de design attendu
Une interface propre et lisible suffit.

Code original
Pas de reprise d'un projet existant tel quel.

07 Rendu attendu

72 h après réception de ce sujet, et **au plus tard la veille de la visio technique à 18h (heure de Paris)**. Le lien est à envoyer en réponse au mail de l'équipe recrutement.

- Un dépôt GitHub public, ou privé avec accès partagé au compte indiqué par l'équipe recrutement. L'API et l'app dans le même dépôt, deux dossiers.
- Un historique Git lisible : plusieurs commits, pas un seul commit final.

- Un README qui contient :
  - les commandes exactes pour lancer l'API, l'app et les tests ;
  - les choix techniques principaux, en quelques lignes ;
  - ce qui n'est pas fait ou pas satisfaisant, et ce qui serait fait avec 2 heures de plus ;
  - les outils d'IA utilisés et pour quoi ;
  - le temps réellement passé.
- Facultatif : une vidéo de 2 minutes maximum, ou quelques captures, montrant le parcours.

08 Critères d'évaluation

Le dernier critère est décisif : **un rendu qui ne peut pas être expliqué ne compte pas.**

Critère	Ce qui est regardé
Fonctionnement	Le projet se lance en suivant le README, le parcours complet marche
Gestion des états	Chargement, erreurs réseau, arrêt du rafraîchissement, aucun crash
Lisibilité du code	Nommage, découpage en composants et fonctions, types TypeScript
API	Routes cohérentes, codes HTTP justes, validation des entrées
Tests	Tests utiles, qui passent, lancés par une commande simple
Git et README	Historique lisible, README honnête et utilisable
Explication	Capacité à expliquer ses choix et à faire évoluer son code en direct

09 La suite : le débrief

Le rendu est discuté pendant la visio technique, sur environ 15 minutes.

5 min
Présentation
Le projet et les choix, app lancée en partage d'écran.

5 min
Questions
Échange sur le code.

5 min
Évolution en direct
Une petite modification, avec les outils habituels, IA comprise.

Une question sur le sujet : répondre au mail de l'équipe recrutement. En cas d'ambiguïté, faire un choix raisonnable et l'écrire dans le README, c'est aussi ce qui est évalué.

Bon courage, et au plaisir d'échanger sur le projet.